Automata and Formal Languages

Notes made by Finley Cooper

13th November 2026

Contents

1 Introduction **3**

2 Regular Languages and Finite-State Automata **3**
2.1 Deterministic Finite-State Automata 3
2.2 Regular Languages 5
2.3 Non-determinism and ε-transitions 5
2.4 Equivalence of DFAs and (ε-)NFAs 7
2.5 Regular Expressions 9
2.5.1 Regular expressions define regular languages 10
2.6 Closure Properties 11
2.7 The Pumping Lemma 12
2.8 Equivalence relations and minimisation of DFAs 12

3 Context Free Grammars **16**
3.1 Grammers and Languages 16
3.1.1 The Chomsky hierarchy 17
3.2 Context free langauges 17
3.3 Chomsky Normal Form 18
3.4 Parse Trees and The Pumping Lemma for Context-Free Languages 20

4 Register Machines and Computatbility **22**
4.1 Register Machines and Computable Functions 23
4.2 Partial Recursive Functions 24
4.3 Partial Computable $\iff$ Partial Recursive 27

1 Introduction

In this course we will have $0 \in \mathbb{N}$.

Definition. (Alphabet) An *alphabet* Σ is a finite non-empty set with elements which are *symbols* or *letters*.

Definition. (String) A *string* or *word* of length m over Σ is $\alpha = x_1 x_2 \dots x_m$ with $x_i \in \Sigma$ or more formally $\alpha \in \Sigma^m$. We write $|\alpha| = m$ for the length of α.

Remark. We will denote the empty word as ε and denote

$$\Sigma^* = \bigcup_{n \in \mathbb{N}} \Sigma^n$$

and $\Sigma^\dagger = \Sigma^* \setminus \{\varepsilon\}$.

Definition. (Concatenation) If $\alpha = x_1 \cdots x_m \in \Sigma^m$ and $\beta = y_1 \cdots y_n \in \Sigma^n$ then $\alpha\beta$ is $x_1 \cdots x_m y_1 \cdots y_n \in \Sigma^{m+n}$.

For $0 \le i \le j \le m$, $x_{i+1} \cdots x_j$ is a substring of α of length $j - i$. So for $\alpha, \beta \in \Sigma^*$, β is a substring of α if and only if there exists $\gamma, \delta \in \Sigma^*$ with $\alpha = \gamma\beta\delta$.

Definition. (Homomorphism) For alphabets Σ, Π, a *homomorphism* $h : \Sigma^* \to \Pi^*$ satisfies $h(\alpha\beta) = h(\alpha)h(\beta)$ for all $\alpha, \beta \in \Sigma^*$.

Remark. This implies that $h(\varepsilon) = \varepsilon$.

Remark. Alternatively we can just consider a function $f : \Sigma \to \Pi^*$ which defines a unique homomorphism extending f by concatenation.

Definition. (Formal Langauge) A *formal language* L over Σ is *any* subset of Σ^*.

2 Regular Languages and Finite-State Automata

2.1 Deterministic Finite-State Automata

Remark. Suppose we want to describe a language L over Σ using a finite amount of information. We could extend Σ with other symbols to get an alphabet $\Pi \supset \Sigma$. and assign a word $w \in \Pi^*$ to L.

We can do this for countably many L. But we want w to *describe* L. We'll now see a countable class of languages where we can do this symbolically. We first give a computational method to determine these languages.

Definition. (Deterministic Finite-State Automata) A *Deterministic Finite-State Automata* (DFA) is a structure

$$D = (Q, \Sigma, \delta, q_0, F)$$

where,

(i) Q is a finite non-empty set of states;
(ii) Σ is an alphabet;
(iii) δ is a transition function from $Q \times \Sigma \to Q$;
(iv) Starting state $q_0 \in Q$;
(v) $F \subseteq Q$ is a set of acceptance states.

The idea for a DFA is that we input a word $w \in \Sigma^*$ we call $w = x_1 \cdots x_n$. The DFA starts in state q_0 and reads the first letter of the input string x_1. The DFA then transitions according to δ to $q_1 = \delta(q_0, x_1) \in Q$. We then repeat this process with x_2 and so on. Once the string is exhusted the string is accepted if $q_n \in F$ otherwise it is rejected. This defines a language.

We describe these with tables and diagrams.

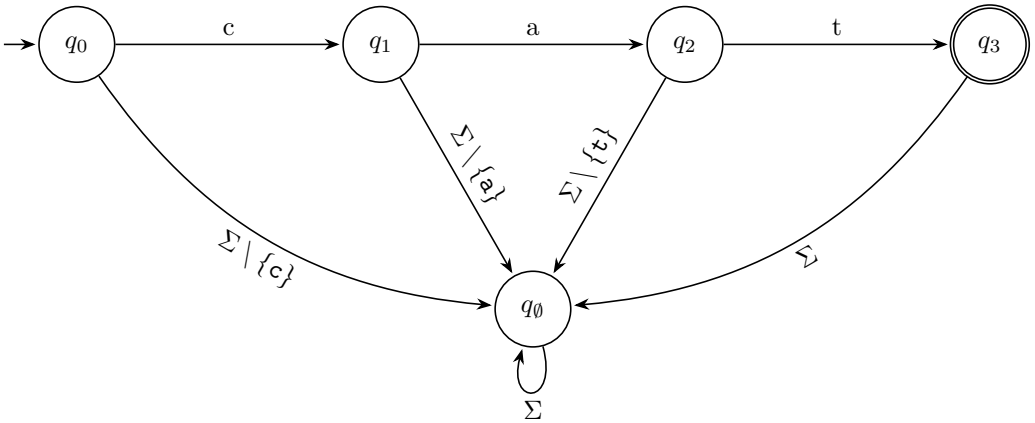
Definition. (Transition diagram) A *transition diagram* for a DFA, D, is a directed labelled graph with:

(i) The vertices being the states Q.
(ii) For each $(q, \sigma) \in Q \times \Sigma$, we draw a directed edge from q to $\delta(q, \sigma)$ and label this edge with σ. So each vertex has $|\Sigma|$ out-edges for every vertex.
(iii) Add an arrow from outside the diagram to q_0 labelled `START`.
(iv) Draw vertex as a circle or as a double circle if the state is an accept state.

If we have lots of edges from one state to another vertex, we can join them in a single edge with the label subset of Σ.

Consider the following diagram which shows a DFA which accepts the word 'cat' and nothing else.

$\Sigma = \{\mathtt{a}, \mathtt{b}, \ldots, \mathtt{z}, _\}$ ($_$ denotes a space)



Definition. (Transition table) For a DFA we can also show the corresponding information in a table. The columns are symbols in Σ and the rows are states $q_0, q_1, \ldots$ and the (i, j) entry is $\delta(q_i, \sigma_j)$.

2.2 Regular Languages

Definition. (Extended transition function) The *extended transition function* of a DFA, D, is

$$\hat{\delta} : Q \times \Sigma^* \to Q$$

defined inductively via

$$\hat{\delta}(q, \varepsilon) = q, \quad \hat{\delta}(q, \sigma_1) = \delta(q, \sigma_1), \quad \delta(q, \sigma_1 \cdots \sigma_k) = \delta(\hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1}), \sigma_k)$$

for all $q \in Q$ and $\sigma_1, \ldots, \sigma_k \in \Sigma$.

So for any word $w \in \Sigma^*$, $\hat{\delta}(q_0, w)$ is the end state with input w.

Lemma. For all $1 \leq \ell \leq k$, we have

$$\hat{\delta}(q, \sigma_1 \ldots \sigma_k) = \hat{\delta}(\hat{\delta}(q, \sigma_1 \ldots \sigma_\ell), \sigma_{\ell+1} \ldots \sigma_k).$$

Proof. Induct on k. We take the base case where $k = \ell$ which follows from the definition. Otherwise,

$$\begin{aligned} \text{LHS} &= \delta(\hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1}), \sigma_k) \\ &= \delta(\hat{\delta}(\hat{\delta}(q, \sigma_1 \ldots \sigma_\ell), \sigma_{\ell+1} \ldots \sigma_{k-1}), \sigma_k) = \text{RHS} \quad \square \end{aligned}$$

Definition. (Regular Language) Let D be a DFA. The language $\mathcal{L}(D)$ of D is the set of words accepted by D namely

$$\mathcal{L}(D) = \left\{ w \in \Sigma^* \mid \hat{\delta}(q_0, w) \in F \right\} \subseteq \Sigma^*$$

A language L is *regular* if there exists a DFA D such that $L = \mathcal{L}(D)$.

2.3 Non-determinism and ε-transitions

We now examine a seemingly more general construction where we can choose which state to move to (or equivalently move to many states simultaneously).

Definition. (Non-deterministic Finite-State Automaton) A *non-deterministic finite-state automaton* (NFA) is a structure

$$N = (Q, \Sigma, \delta, q_0, F)$$

just as a for a DFA, however δ is now a map $Q \times \Sigma \to \mathcal{P}(Q)$, to contain the idea of moving

to many states "at once".

An NFA as input $w = \sigma_1 \dots \sigma_k \in \Sigma^*$ while in state q_0, reads σ_1 then "moves to" all of the states in $\delta(q_0, \sigma_1)$. Then for all states $p \in \delta(q_0, \sigma_1)$, read σ_2 and move to $\sigma(p, \sigma_2)$. Continue to end of w to get to all end states $E_w \subseteq Q$. If $E_w \cap F \neq \emptyset$, then w is accepted, otherwise w is rejected.

Remark. Note that since $\emptyset \in \mathcal{P}(Q)$ we can have $\delta(p, \sigma) = \emptyset$. But what if we're currently in states $S \subseteq Q$ and to read σ_i only to find that $\bigcup_{p \in S} \delta(p, \sigma_i) = \emptyset$. Then we are in "no state" and we are stuck in NFA limbo until the end of the word, where the word is automatically rejected.

Given an NFA N, we can define the transition diagram and the transition table in the same way as for a DFA although the table now contains subsets of Σ and the diagram has multiple directed edges for a single letter in Σ.

Definition. (Extended transition function) For an NFA the *extended transition function* $\hat{\delta}$ of N is $\hat{\delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$ defined by $\hat{\delta}(q, \varepsilon) = \{q\}$ for $q \in Q$, $\hat{\delta}(q, \sigma) = \delta(q, \sigma)$ for $q \in Q$ and $\sigma \in \Sigma$ and

$$\hat{\delta}(q, \sigma_1 \dots \sigma_k) = \bigcup_{p \in \hat{\delta}(q, \sigma_1 \dots \sigma_{k-1})} \delta(p, \sigma_k).$$

So for $w \in \Sigma^*$ we have that $\hat{\delta}(q_0, w)$ is the set of end states E_w.

We want to define a more general type of state machine. We want a more flexible type where we can change certain states without reading a symbol. We think about this as reading ε between the letters.

Definition. (ε-NFA) An *ε-NFA* is a structure $E = (Q, \Sigma, \delta, q_0, F)$ defined just as for an NFA except

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \to \mathcal{P}(Q)$$

where $\varepsilon \notin \Sigma$ but is acting like a symbol.

Definition. (ε-closure) For an ε-NFA, E, and a state $q \in Q$, we define the *ε-closure* $\text{ecl}(q)$ of q as the set of states reached from q by repeated ε-transitions.

That is define inductively, $S_0(q) = \{q\}$, $S_1 = \{q\} \cup \delta(q, \varepsilon)$ and $S_{i+1}(q) = S_i(q) \cup \bigcup_{r \in S_i(q)} \delta(r, \varepsilon)$. Then we can say that

$$\text{ecl}(q) = \bigcup_{i=0}^{\infty} S_i(q) = S_N(q)$$

for some N since we have a strictly increasing chain which balances at some point. We can also take the ε-closure of a subset $S \subseteq Q$ defined as $\text{ecl}(S)$, defined as the union of the ε-closure of each element in S. We have that the ε-closure moves in and out of unions.

Definition. (ε-closed) We say that $R \subseteq Q$ is *ε-closed* if $\text{ecl}(R) = R$.

Lemma. For any $S \subseteq Q$ we have that $\text{ecl}(S)$ is ε-closed.

Proof. Let $C = \text{ecl}(S)$. We need that $\text{ecl}(C) \subseteq C$. But $x \in \text{ecl}(C)$ means that $x \in \text{ecl}(r)$ for some $r \in C$, so there exists a finite sequence of ε-transitions from r to x. But $r \in \text{ecl}(S)$ so there exists some $s \in S$ with a sequence of ε-transitions from s to r. Putting these together we get that $x \in S_{m+n}(s) \subseteq \text{ecl}(s) \subseteq C$, where m and n are the lengths of the ε-transitions of the paths from r to x and r to s respectively. □

An ε-NFA has input $w = \sigma_1 \cdots \sigma_k \in \Sigma^*$. From q_0 it moves to all states in $\text{ecl}(q_0)$ and only then reads σ_1. For each $x \in \text{ecl}(q_0)$ it takes the set of the states $\delta(x, \sigma_1)$ and for each $y_x \in \delta(x, \sigma_1)$ it now goes to $\text{ecl}(y_x)$, so now in these states unioned over y_x and x. Continue with σ_2 and repeat, until we read the last letter σ_k and then w is accepted if any of these end states are in F, otherwise it is rejected.

Definition. (Extended transition function) For an ε-NFA, its *extended transition function* is a function

$$\hat{\delta} : Q \times \Sigma^* \to \mathcal{P}(Q)$$

defined inductvely on the length of the word $w = \sigma_1 \cdots \sigma_k$ via $\hat{\delta}(q, \varepsilon) = \text{ecl}(q)$.

$$\hat{\delta}(q, \sigma) = \text{ecl}\left(\bigcup_{p \in \text{ecl}(q)} \delta(p, \sigma) \right), \quad \hat{\delta}(q, \sigma_1 \ldots \sigma_k) = \text{ecl}\left(\bigcup_{p \in \hat{\delta}(q, \sigma_1 \ldots \sigma_{k-1})} \delta(p, \sigma_k) \right).$$

What is the relationship between these finite-state machines?

(i) Any DFA is a NFA (by changing the transition function to map to the singleton set insteaed).

(ii) Any NFA is an ε-NFA by defining $\delta(q, \varepsilon) = \emptyset$.

However all of these state machines actually define the same subset of languages called *regular languages*! We'll work up to a proof of that now.

Definition. (Language of NFAs and ε-NFAs) Given an NFA or a ε-NFA, E, it's language $\mathcal{L}(E)$ is given by

$$\mathcal{L}(E) = \left\{ w \in \Sigma^* : \hat{\delta}(q_0, w) \cap F \neq \emptyset \right\}.$$

2.4 Equivalence of DFAs and (ε-)NFAs

Any regular language is the language of an (ε-)NFA which is fairly simple to see. We want ot show the converse using the *subset construction*

First suppose we have an NFA, N. We produce a DFA D on the same alphabet Σ such that $\mathcal{L}(N) = \mathcal{L}(D)$.

Definition. (Subset construction) For the NFA $N = (Q, \Sigma, \delta, q_0, F)$, the *subset construc-*

tion for N produces the DFA

$$D = (\mathcal{P}(Q), \Sigma, \Delta, \{q_0\}, \mathcal{F})$$

where $\mathcal{F} = \{S \subseteq Q : S \cap F \neq \emptyset\}$ and for $S \subseteq Q, \sigma \in \Sigma$ we have that

$$\Delta(S, \sigma) = \bigcup_{p \in S} \delta(p, \sigma) \in \mathcal{P}(Q).$$

Theorem. For the NFA N and DFA D obtained from the subset construction, let $\hat{\delta}$ and $\hat{\Delta}$ to be their respective extended transition functions. Then for all $w \in \Sigma^*$ we have that

$$\hat{\Delta}(\{q_0\}, w) = \hat{\delta}(q_0, w).$$

Proof. Induct on word length $k = |w|$. The base case $k = 0$ is $w = \varepsilon$ then the LHS $= \{q_0\} =$ RHS. Now the inductive hypothesis

$$\hat{\Delta}(\{q_0\}, \sigma_1 \dots \sigma_{k-1}) = \hat{\delta}(q_0, \sigma_1 \dots \sigma_{k-1}).$$

Then for $w = \sigma_1 \dots \sigma_k$, we have that

$$\begin{aligned}\hat{\delta}(q_0, w) = \bigcup_{p \in \hat{\delta}(q_0, \sigma_1 \dots \sigma_{k-1})} \delta(p, \sigma_k) &= \Delta(\hat{\delta}(q_0, \sigma \dots \sigma_{k-1}), \sigma_k) \\ &= \Delta(\hat{\Delta}(\{q_0\}, \sigma_1 \dots \sigma_{k-1}), \sigma_k) = \hat{\Delta}(\{q_0\}, \sigma_1 \dots \sigma_k) \quad \square\end{aligned}$$

Corollary. A language L is accepted by some DFA if and only if it accepted by some NFA.

Proof. Forwared direction is obvious. In the converse, suppose we're given an NFA N, form a DFA D from the subset construction. Then w is accepted by N if and only if $\hat{\Delta}(\{q_0\}, w) \in \mathcal{F}$ by the previous theorem. $\square$

Definition. (Subset construciton) For the ε-NFA, $E = (Q, \Sigma, \delta, q_0, F)$ the *subset construction* with ε-transitions produces the DFA $D(\mathcal{P}(Q), \Sigma, \Delta, \text{ecl}(\{q_0\}), \mathcal{F})$ where $\mathcal{F} = \{S \subseteq Q | S \cap F \neq \emptyset\}$ as before and for $S \subseteq Q, \sigma \in \Sigma$ we have that

$$\Delta(S, \sigma) = \text{ecl}\left(\bigcup_{p \in S} \delta(p, \sigma)\right) \in \mathcal{P}(Q).$$

Remark. This can be put into words as the subsets of states reached from S by following an edge labelled σ then some edges labelled ε.

As before we have a theorem proving this constructions defines the same language.

Theorem. For an ε-NFA, E, and DFA, D, obtained from the subset construction, we have that

$$\hat{\Delta}(\text{ecl}(\{q_0\}), w) = \hat{\delta}(q_0, w).$$

Proof. As in the previous theorem, the base case for $w \in \varepsilon$ gives $\text{ecl}(\{q_0\})$. Then the inductive steps is similar to the previous proof with the ecl places and handled with appropiately. □

Corollary. A language L is accepted by some DFA if and only if it is accepted by some NFA if and only if it is accepted by some ε-NFA.

2.5 Regular Expressions

We know now that DFAs and (ε-)NFAs defnie the same languages. We knwo want to give an algebraic or symbolic description of a regular language: that of a regular expression. Then we will show that DFAs $\implies$ Regular Expressions $\implies$ ε-NFAs, which will show they're all equivalent.

Definition. (Creating new languages from old)

(i) The *union* of languages L, M over Σ_L, Σ_M is the language $L \cup M$ over $\Sigma_L \cup \Sigma_M$.

(ii) The *concatenation* LM of languages L, M is the words $w = uv$ where $\{(u, v) \in L \times M\} \subseteq (\Sigma_L \cup \Sigma_M)^*$.

(iii) The *closure* L^* of a language L is a set of all words formed by concatenating an arbitrary number of wordes in L so

$$L^* = \bigcup_{n \geq 0} L^n$$

including the empty word.

Remark. For $n \geq 2$ we can have $L \not\subseteq L^n$ if $\varepsilon \notin L$, but not if $\varepsilon \in L$ since we can use the empty word to pad the word out. However certainly $L \subseteq L^*$.

Definition. (Regular Expression) A *regular expression* R is a finite string of symbols denoting a language $\mathcal{L}(R)$ over some alphabet, build up from the following inductive rules:

(i) Both ε and $\emptyset$ are regular expressions with $\mathcal{L}(\varepsilon) = \{\varepsilon\}$ and $\mathcal{L}(\emptyset) = \emptyset$.

(ii) If a is a symbol then a is a regular expression with $\mathcal{L}(a) = \{a\}$.

(iii) If E and F are regular expressions then $E + F$ is a regular expression denoting union so $\mathcal{L}(E + F) = \mathcal{L}(E) \cup \mathcal{L}(F)$.

(iv) EF is a regular expression denoting concatenation, so $\mathcal{L}(EF) = \mathcal{L}(E)\mathcal{L}(F)$.

(v) E^* is a regular expression denoting the closure of E so $\mathcal{L}(E^*) = \mathcal{L}(E)^*$.

(vi) (E) is a regular expression with the same language of E and brackets are used to avoid ambiguity and enforce order of operations.

Definition. (Order of operations) The order of precendence for regular expressions is brackets, then Kleene star, then concatenation, then addition.

Example. For example the regular expression $01^* + 1$ defines the language accepting 1 or anything of the form 0(any number of 1s).

2.5.1 Regular expressions define regular languages

Theorem. For each regular expression R, there exists an ε-NFA, E, such that $\mathcal{L}(E) = \mathcal{L}(R)$.

Proof. We build E up by induction. Our ε-NFA will be constructed such that

(i) There is a unique accepting state;

(ii) There are no directed edges into the start state except `START`;

(iii) No directed edges out of the accepting state.

We can construct ε-NFAs for the regular expressions ε, $\emptyset$ and a just by taking two states (with the first one the starting state and the second state accepting) and joining them by ε, nothing, and a respectively.

Now assume for a regular expression R we have an ε-NFA E with $\mathcal{L}(E) = \mathcal{L}(R)$ with the construction properties as above. Now suppose we're given two regular expressions R and S each with their own ε-NFAs of the form we like. Then we can joint the ε-NFAs together by adding a new start state with ε-transitions to the two NFAs start states and then connect the NFAs end states to a new accept state also with an ε-transition. This accepts the language $\mathcal{L}(R + S)$ exactly.

Similarly for RS we can construct a ε-NFA by doing R then S connnected by an ε-transition.

Finially we can create an ε-NFA for the Kleene star, by connecting the start and end state to eachother with ε-transitions. □

Theorem. For each DFA D there is a regular expression R such that $\mathcal{L}(D) = \mathcal{L}(R)$.

Proof. Given $D = (Q, \Sigma, \delta, q_0, F)$, label the states by $\{1, 2, \ldots, n\}$ and let the start state be 1. Now given $1 \leq i, j \leq n$ and $0 \leq k \leq n$ we construct by induction on k a regular expression $R_{ij}^{(k)}$ with language exactly the words w that begin at state i, end at state j and are such that all the intermediate states pass through (not including the endpoints) are at most k (by label not by count). So we're thinking of w as a directed path from i to j.

For $k = 0$ we have to go directly from i to j. If $i \neq j$ then set

$$R_{ij}^{(0)} = \begin{cases} a_1 + \cdots + a_\ell & \ell > 0 \\ \emptyset & \ell = 0 \end{cases}$$

and if $i = j$ then let

$$R_{ii}^{(0)} = \begin{cases} a_1 + \cdots + a_\ell + \varepsilon & \ell > 0 \\ \varepsilon & \ell = 0 \end{cases}$$

where we're using ε to denote not moving. Suppose that that $R_{ij}^{(t)}$ is defined for all $t < k$. Give a path π from i to j not passing through states $k + 1, k + 2, \ldots, n$. Either

(i) If π does not pass through k either, the word which would read from π is already in $\mathcal{L}(R_{ij}^{(k-1)})$ and vice-versa.

(ii) Otherwise π does pass through k various times, so we split the path into $\sigma, \pi_1, \pi_2, \ldots, \pi_r, \omega$ where σ goes from i to k, hitting k for the first time on the last step, π_ℓ is a loop from k to k and ω is a path from k to j which doesn't hit k (except at the start). Now the word read from σ is in $\mathcal{L}(R_{ik}^{(k-1)})$ and then each word $\pi_1, \ldots, \pi_r$ is in $\mathcal{L}(R_{kk}^{(k-1)})$ and ω is in $\mathcal{L}(R_{kj}^{(k-1)})$. The the word read from π is in the language given by the regular expression

$$R_{ik}^{(k-1)} {R_{kk}^{(k-1)}}^* R_{kj}^{(k-1)}$$

This for cases (i) and (ii) we have $\mathcal{L}(R_{ij}^{(k-1)} + R_{ik}^{(k-1)} {R_{kk}^{(k-1)}}^* R_{kj}^{(k-1)})$. We therefore define the regular expression as $R_{ij}^{(k)}$. Finally if $F = \{j_1, \ldots, j_s\}$ then

$$\mathcal{L}(D) = \mathcal{L}(R_{1j_1}^{(n)} + \cdots + R_{1j_s}^{(n)}).$$

So we're done. □

2.6 Closure Properties

Theorem. Let L, M be regular languages over Σ_L, Σ_M. Then
(i) $L \cup M$ is regular over $\Sigma_L \cup \Sigma_M$;
(ii) LM is regular;
(iii) L^* is regular over Σ_L.

Proof. We know a language is regular if and only if it is represented by a regular expression. Let R_L, R_M be regular expressions for L and M. Then we can prove as follows:

(i) $R_L + R_M$ clearly works;

(ii) $R_L R_M$ clearly works;

(iii) R_L^* clearly works.

These can be checked trivally. □

Theorem. Let L be a regular language over Σ then so is

$$\overline{L} = \Sigma^* \setminus L$$

called the complement of L.

Proof. Let D be a DFA for L. Then modify the DFA so every accept state is not longer an accept state and every non-accept state is an accept state, so replace F with $Q \setminus F$. This is a DFA which clearly accepts the langauge $\overline{L}$. □

Corollary. If L, M are regular languages then so is $L \cap M$.

Proof. Use De Morgan's law. Since $\overline{L}$ and $\overline{M}$ are regular, then so is $\overline{L} \cup \overline{M}$. Hence the complement of this is just $L \cap M$ by De Morgan's law. □

2.7 The Pumping Lemma

How do we show a language is *not* regular? We know that

(i) Any finite language is regular;

(ii) As Σ^n is a finite set, if L is infinite then for all N there exists a word $w \in L$ with $|w| \geq N$.

Suppose a word w is accepted by a DFA D but $|w| \geq |Q|$. Then w defines a directed path in a transition diagram I'_D which must visit some state at least twice. So we can write $w = xyz$ for subwords x, y, z where y gives us a loop and then go around the loop again or not at all.

Theorem. (Pumping Lemma) If L is a regular language then there exists a $n \geq 1$ called the *pumping number* for L such that for all words $w \in L$ with $|w| \geq n$, we can write $w = xyz$ for subwords x, y, z where
(i) $y \neq \varepsilon$;
(ii) $|xy| \leq n$;
(iii) $\forall k \geq 0$ we have the word $xy^k z \in L$.
i.e we can pump appearance of y.

Proof. We have that $L = \mathcal{L}(D)$ for a DFA $D = (Q, \Sigma, \delta, q_0, F)$ and set $n = |Q|$. Now take an accepted word $w \in L$ with $w = \sigma_1 \dots \sigma_m$ where $m \geq n$. Let $p_i = \hat{\delta}(q_0, \sigma_1 \dots \sigma_i)$: the state we're in after reading the first i letters of w with $0 \leq i \leq m$. By the pigeonhole principle, we must have $0 \leq \ell < r \leq n$ with $p_\ell = p_r$. So we can write

$$w = \underbrace{\sigma_1 \dots \sigma_\ell}_{x} \underbrace{\sigma_{\ell+1} \dots \sigma_r}_{y} \underbrace{\sigma_{r+1} \dots \sigma_m}_{z} = xyz$$

thus we have that $|xy| = r \leq n$ and since $\ell < r$ we have that $y \neq \varepsilon$. So for any $k \geq 0$, let's input $xy^k z$ into D. If $k = 0$ then we can see that this is accepted since we just don't "do the loop" which y does. Similarly for $k > 1$, the route corresponds to just doing the y loop k times which is obviously still accepted. □

Example. If L is regular and infinite then the pumping lemma applies to some word $w \in L$ so $\{|w| : w \in L\}$ contains an arithmetic progression. Hence the language $\{a^n : n \text{ is prime}\}$ is not regular.

Example. The language $L = \{0^i 1^i : i \geq 0\}$ is not regular. Suppose that L is regular. Take $w = 0^n 1^n \in L$ for the n in the pumping lemma and write $w = xyz$ so $xy = 0^m$ and y^ℓ for some $0 < \ell \leq m \leq n$. Now we can pump down, so $xz = 0^{n-\ell} 1^n$ which clearly isn't in the language, but the pumping lemma says it is in the language. Alternatively we can pump up, so $xy^2 z = 0^{n+\ell} 1^n$ which also clearly isn't in the language, but again the pumping lemma say it's in the language so L isn't regular.

This exact argument words for $L' = \{w : w \text{ contains same number of 0s and 1s}\}$.

2.8 Equivalence relations and minimisation of DFAs

Given a DFA D over an alphabet Σ, we want a DFA D' over Σ with $\mathcal{L}(D') = \mathcal{L}(D)$ but with the smallest possible number of states. DFAs from the subset construction often have an excess of states.

Definition. (Accessible state) An *accessible state* $q \in Q$ of a DFA is one where $\exists w \in \Sigma^*$ with $\hat{\delta}_D(q_0, w) = q$. Otherwise we say that the state is *inaccessible*.

From D we can form a DFA A with no inaccessible states and $\mathcal{L}(A) = \mathcal{L}(D)$ if $n = |Q_D|$ then form $S_i \subseteq Q_D$ via

$$S_0 = \{q_0\}, \quad S_{i+1} = \bigcup_{q \in S_i} \left(\bigcup_{\sigma \in \Sigma} \delta_D(q, \sigma) \right)$$

so S_i is states reachable from q_0 after i steps. But any accessible state can be reached after $n-1$ steps (remove closed loops of repeated states) so set

$$S = \bigcup_{j=0}^{n-1} S_j$$

and $Q_A = S, F_A = F_D \cap S$ and δ_A is a restriction of δ_D to $Q_A \times \sigma$ maps to Q_A by construction.

Remark. This might not be enough. Moreover what about uniqueness?

Definition. (Equivalence) For A, B both DFAs over Σ we say that A and B are *equivalent*, written $A \equiv B$ if (up to relabling of non-start states) they are the same DFA.

Remark. Obviously this means they define the same language too.

Now for a single DFA, we say group together equivalent states.

Definition. (Equivance of states) For a DFA, D, we call two states, $p, q \in Q$ *equivalent* or *indistinguishable* if $\forall w \in \Sigma^*$ we have that

$$\hat{\delta}(p, w) \in F \iff \hat{\delta}(q, w) \in F$$

written as $p \sim q$, or otherwise p, q are *distinguished* by w.

Proposition. For a DFA, D and $p, q \in Q$ we have that
(i) $\sim$ is an equivalence relation on Q;
(ii) If $p \sim q$ then $\forall \sigma \in \Sigma$ we have that $\delta(p, \sigma) \sim \delta(q, \sigma)$ and $\hat{\delta}(p, w) \sim \hat{\delta}(q, w)$ for all words $w \in \Sigma^*$;
(iii) If $p \sim q$ then p is accepting if and only if q is.

Proof.

(i) Trivally symmetric and reflexive. For transitivity suppose that $p \sim q$ and $q \sim r$ and take a word $w \in \Sigma^*$. Then both $\hat{\delta}(p, w)$, $\hat{\delta}(q, w)$ are either both accepting or both not accepting as are both of $\hat{\delta}(q, w)$ and $\hat{\delta}(r, w)$ so true for $\hat{\delta}(p, w)$ and $\hat{\delta}(r, w)$.

(ii) Suppose we have $\sigma \in \Sigma$ with $\delta(p, \sigma) \not\sim \delta(q, \sigma)$ so *wlog* we have $w \in \Sigma^*$ with $\hat{\delta}(\delta(p, \sigma), w)$ accepted but $\hat{\delta}(\delta(q, \sigma), w)$ is not. Then p, q are distinguished by σw which is a contradiction since $p \sim q$ now induction on $|w|$.

(iii) Just put $w = \varepsilon$ in the definition. □

We now reduce the number of states in a DFA by identifying as follows.

Definition. (Minimal DFA) Given a DFA, $D = (Q, \Sigma, \delta, q_0, F)$ we define the *minimal DFA*, $D/\sim$ for D as $(Q/\sim, \Sigma, \delta, [q_0]_\sim, F')$ where

$$\delta'([q], \sigma) = [\delta(q, \sigma)]$$

which is well-defined by (ii) from the previous proposition and the accept states

$$F' = \{[p] \mid p \in F\}$$

which works by (iii) from the proposition.

Lemma. For a DFA D and $D/\sim$ as above, take $q \in Q$ and $w \in \Sigma^*$. Then

$$\hat{\delta}'([q], w) = [\hat{\delta}(q, w)].$$

Proof. Proceed by induction on $|w|$. b

$$\begin{aligned}\hat{\delta}'([q], \sigma_1 \dots \sigma_k) &= \delta'(\hat{\delta}'([q], \sigma_1 \dots \sigma_{k-1}), \sigma_k) \\ &= \delta'([\hat{\delta}(q, \sigma_1 \dots \sigma_{k-1})], \sigma_k) = [\delta(\hat{\delta}(q, \sigma_1 \dots \sigma_{k-1}), \sigma_k)] = [\hat{\delta}(q, w)] \quad \square\end{aligned}$$

Theorem. For a DFA D and its minimal DFA $D/\sim$, we have that

$$\mathcal{L}(D/\sim) = \mathcal{L}(D).$$

Proof. For all $w \in \Sigma^*$ we have that

$$\begin{aligned}w \in \mathcal{L}(D/\sim) &\iff \hat{\delta}'([q_0], w) \in F' \\ &\iff [\hat{\delta}(q_0, w)] \in F' \\ &\iff \hat{\delta}(q_0, w) \in F \\ &\iff w \in \mathcal{L}(D). \quad \square\end{aligned}$$

Lemma. For a DFA D and its minimal DFA $D/\sim$, no two distinct states of $D/\sim$ are equivalent.

Proof. Suppose for two states p, q of D we have $[p] \sim [q]$ in $D/\sim$. Then for all words w, we have $\hat{\delta}'([p], w), \hat{\delta}'([q], w)$ are both accepting or non-accepting in $D/\sim$. So by previous lemma $[\hat{\delta}(p, w)]$, $[\hat{\delta}(q, w)]$ are both accepting or non-accepting, so $\hat{\delta}(p, w), \hat{\delta}(q, w)$ are both accepting or non-accepting in D. Thus $p \sim q$ in D and so $[p] = [q]$ in $D/\sim$. $\square$

Corollary. If D is a DFA then $(D/\sim)/\sim \equiv D/\sim$ as DFAs.

Proof. On forming $D/\sim$, no two distinct states of $D/\sim$ are equivalent so for each state $s \in D/\sim$ on forming $(D/\sim)/\sim$ we see that these states are $[s] = \{s\}$ so on relabelling back to s we find that we have some accept states and transition function for $D/\sim$ and $(D/\sim)/\sim$. $\square$

Example. Consider the "cat" DFA from before. This has no inaccessible states and no equivalent states. Now add a "phantom" state q_4 to C with no arrows to or from the other states and which is not an accept state then in this new DFA D we have that q_4 is inaccessible. Moreover $\hat{\delta}(q_4, w) \notin F = \{q_3\}$ so q_4. Thus all states of D are inequivalent. So $D/\sim$ is D but clearly not the smallest, since we can get rid of q_4.

Here we will first remove inaccessible states, and then minimise with the equivalence.

Theorem. If the DFA D has no inaccessible states and A is a DFA with $\mathcal{L}(D) = \mathcal{L}(A)$ then
(i) A has at least as many states as $D/\sim$;
(ii) If A and $D/\sim$ have the same number of states then $A \equiv D/\sim$.

Proof. Take a DFA B with the fewest states, subject to $\mathcal{L}(B) = \mathcal{L}(D)$. Then B as no inaccessible states and $D/\sim$ also has none since for any state $[q]$ we have some w where $\hat{\delta}(q_0, w) = q$ since it holds for D, then $\hat{\delta}'([q_0], w) = [q]$. So we have that $D/\sim = (Q/\sim, \Sigma, \delta', [q_0], F')$ and $B = (S, \Sigma, d, S_0, G)$.

We form the "disjoint union DFA" U of $D/\sim$ and B where

$$U = (Q/\sim \sqcup S, \Sigma, \Delta, [q_0], G \sqcup F')$$

where

$$\Delta : (Q/\sim \sqcup S) \times \Sigma \to Q/\sim \sqcup S$$

is defined via δ' and d. As $\mathcal{L}(D/\sim) = \mathcal{L}(D) = \mathcal{L}(B)$, this means that states $[q_0]$ and s_0 of U are equivalent because reading a word in U start at $[q_0]$ is the same as it is in $D/\sim$ and also in B starting at s_0. Then for any state $p \in Q/\sim$, we have a state $s \in S$ with $p \sim s$ in U because we have some $w \in \Sigma^*$ with $\hat{\delta}([q_0], w) = p$ as it's accessible in $D/\sim$ but $[q_0] \sim s_0$ in U which implies that $p = \hat{\Delta}([q_0], w) \sim \tilde{\Delta}(s_0, w)$ in $U = \hat{d}(s_0, w) = s \in S$. Now consider in U. The equivalence classes of the states if $|D/\sim| > |B|$ then have two states p_1, p_2 of $D/\sim$ which are equivalent to a single state of B so they are equivalent to eachother in U, i.e we have that

$$\hat{\Delta}(p_1, w) \in G \sqcup F' \iff \hat{\Delta}(p_2, w) \in G \sqcup F'.$$

But by disjointness of the DFAs in U, we have that $\hat{\Delta}(p_2, w) \in G \sqcup F' \iff \hat{\delta}'(p_2, w) \in F'$ so p_1, p_2 are equivalent states in $D/\sim$ which is a contradiction. So now we have $|D/\sim| = |B|$ by minimality and each state of $D/\sim$ is equivalent in U to a state in B and visa-versa. As pairs of equivalent statesin U ar preserved under Δ and all states are accessible, the DFAs $D/\sim$ and B are "matched" exactly under the equivalence, i.e $[q_0] \sim s_0$ in U so $\delta'([q_0], \sigma) = \Delta([q_0], \sigma) \sim \Delta(s_0, \sigma) = d(s_0, \sigma)$ so since σ is arbitrary, they are the same DFA upto a relabling of states. □

We'll now describe an algorithm to minimise a DFA. First remove inaccessible states. To find equivalent states, use *Hopcroft's table filling algorithm.* Having the table $Q \times Q$ with the (p, q)th entry having a $\times$ in it to show that $p \not\sim q$. Only fill in the lower left part (since we have symmetry) and start with all blanks.

For the base step put a $\times$ whenver we have (p, q) with one in F and one not (since ε distinguishes). For the inductive step take any pair (p, q) which is empty. If we have $\sigma \in \Sigma$ with $\delta(p, \sigma) = r$, $\delta(q, \sigma) = s$ and (r, s) marked $\times$. Then p, q are distinguished by σ concatenated with whatever distinguishses (r, s). Halt when this fails for all unmarked pairs and symbols. The algorithm works of we have a pair (p, q) unmarked by $p \not\sim q$, take one with the shortest distinguishing

word $w = \sigma_1 \dots \sigma_n$ where we say $\hat{\delta}(p, w) \in F, \hat{\delta}(q, w) \notin F$. Now $w \neq \varepsilon$ (by the base step), so $r = \delta(p, \sigma_1), s = \delta(q, \sigma_1)$ distinguished by $\sigma_2 \dots \sigma_n$, shorter word so (r, s) marked with a $\times$ so (p, q) would be marked in the inductive step. Finally, given 2 DFAs D_1, D_2 over Σ we can test whether or not they have the same language. First we do the first two steps above to get E_1, E_2 then $E_1/\sim, E_2/\sim$ which the same languages as D_1 and D_2 respectively. So $\mathcal{L}(E_1/\sim) = \mathcal{L}(E_2/\sim)$ i.e the states are a relabling of eachother iff $\mathcal{L}(D_1) = \mathcal{L}(D_2)$.

3 Context Free Grammars

3.1 Grammers and Languages

Definition. (Production rule) Given an alphabet Ω, a *production* or *rewrite* rule is an element $(\alpha, \beta) \in \Omega^+ \times \Omega^*$, written $\alpha \longrightarrow \beta$, read as "replace α with β". Think of this as if $w \in \Omega^*$ contains α as a substring, so $w = \gamma\alpha\delta$ then we can *choose* to replace α with β, changing w to $\gamma\beta\delta$.

Remark. We have that $\alpha \neq \varepsilon$ but we can have $\alpha \longrightarrow \varepsilon$ called an ε-production.

Definition. (Rewrite system) A *rewrite system* (Ω, P) is an alphabet Ω and a finite set P of rewrite rules.

Definition. (Grammar) A (variable based) *grammar* $G = (N, \Sigma, P, S)$ is a rewrite system where $\Omega = \Sigma \sqcup N$ is divided into Σ the *terminal* letters and N the *non-terminal* letters. Any production rule $\alpha \to \beta \in P$ has $\alpha \in N^+$ (can't replace terminals). $S \in N$ is the start symbol.

Remark. We usually use capitals for non-terminals and lower case for terminals. We use lower case greek for words over capital greek alphabets.

We need to use the grammar to construct a language. For a grammar $G = (N, \Sigma, P, S)$ and $\alpha, \beta \in \Omega^*$ we say that

(i) β is derivable from α in one step if $\alpha = \alpha_1\gamma\alpha_2$, $\beta = \alpha_1\delta\alpha_2$ and there exists a production rule $(\gamma, \delta) \in P$, written as
$$\alpha \xrightarrow[G]{1} \beta.$$

(ii) A *derivation* of length n of β from α written as $\alpha \xrightarrow[G]{n} \beta$ which means that we have a γ with $\alpha \xrightarrow[G]{n-1} \gamma, \gamma \xrightarrow[G]{1} \beta$ and $\alpha \xrightarrow[G]{0} \alpha$.

(iii) $\alpha \xrightarrow[G]{*} \beta$ means that there exists an n with $\alpha \xrightarrow[G]{n} \beta$ and we say that β is *derivable* from α.

(iv) A word $w \in \Omega^*$ is derivable from the start symbol S is in *sequential form.*

(v) A word in sequential form is a *sentence* if $w \in \Sigma^*$ (only contains terminals).

Definition. The language $\mathcal{L}(G)$ generated by a grammar G is

$$\mathcal{L}(G) = \left\{ w \in \Sigma^* \mid S \xrightarrow[G]{*} w \right\}$$

Remark. In general a word $w \in \mathcal{L}(G)$ could have many derivations from S of same or even different lengths. We can also have grammars which are distinct but define the same language (i.e different production rules).

3.1.1 The Chomsky hierarchy

Suppose that $\Omega = N \sqcup \Sigma$.

Definition. (Chomsky Hierarchy) A production rule $(\alpha, \beta) \in \Omega^+ \times \Omega^*$ is of
(i) Type 0: If $\alpha \in N^+$;
(ii) Type 1: Noncontracting if $\alpha \in N^+$ and if $|\alpha| \leq |\beta|$;
(iii) Type 2: If $\alpha = A \in N$ and $|\beta| \geq 1$;
(iv) Type 3: If $\alpha = A \in N$ and $\beta = a$ or $\beta = aB$ for $B \in N$, $a \in \Sigma$.
We say that a grammar is of type n if all of its production rules are of type n.
A language L is of type n if there exists a grammar G of type n with $L = \mathcal{L}(G)$.

For a type 1 grammer suppose that

$$\beta_0 = S \xrightarrow[G]{1} \beta_1 \xrightarrow[G]{1} \cdots \xrightarrow[G]{1} \beta_m = w$$

of minimum length m. But $|\beta_i| \leq |\beta_{i+1}|$ and if $\ell = |\beta_i| = \cdots = |\beta_{i+j}|$ for $j \geq |\Omega|^l$ then 2 of these βs are the same, so m is not minimum. Then

$$m \leq \sum_{i=1}^{|w|} |\Omega|^\ell = M$$

so we can tell whether or not $w \in L = \mathcal{L}(G)$ systematically by creating all derivations of length $\leq M$ from S if we see w then *yes* if not then *no*.

Remark. If $\varepsilon \in \mathcal{L}(G)$ then we need some ε-production $\alpha \to \varepsilon$ so languages of type 1 (and 2 and 3) do not contain ε.

3.2 Context free langauges

Definition. (Context free grammar) A *context free grammar* (CFG) $G = (N, \Sigma, P, S)$ is where every production rule is of the form $(A, \alpha) \in N \times \Omega^*$ (so type 2 but maybe also with ε-productions).

Definition. (Context free language) A language L is *context free* (CFL) if $L = \mathcal{L}(G)$ for some CFG G.

Remark. We write $A \to a \mid b$ to mean the two production rules $A \to a, A \to b$.

Example.

(i) A type 2 language is a context free language;

(ii) Suppose that $N = \{S\}$, $\Sigma = \{a, b\}$ with production rules $S \longrightarrow aSb \mid \varepsilon$. This generates the language $\{a^n b^n : n \geq 0\}$ which we know is not regular by the pumping lemma.

(iii) Regular languages are context free languages (Example Sheet 2, Questions 8 and 9).

(iv) If we have the production rules $S \longrightarrow SaSbS \mid SbSaS \mid \varepsilon$ which gives us a language which has the same number of a's and b's.

(v) The langauge X which is the set of words which are palindromes is a context free language since (if we just consider $\Sigma = \{a, b\}$) it is generated by the rules $S \to aSa \mid bSb \mid a \mid b \mid \varepsilon$.

Definition. (Language of balanced parentheses) Let $\Sigma = \{(,)\}$. For $w \in \Sigma^*$ let $L(w)$ be the number of left brackets in w and let $R(w)$ be the number of right brackets in w. Then w is balanced if $L(w) = R(w)$ and for all $x \in \Sigma^*$ with $w = xy$ we have that $L(x) \geq R(x)$.

This language can be shown to be context free, however it is not regular.

3.3 Chomsky Normal Form

We want to be able to convert any context free grammar into a standard form. This standard form may be longer, but should have simplier production rules and define the same language (except will be missing ε).

Definition. (Chomsky normal form) A CFG $G = (N, \Sigma, P, S)$ is in *Chomsky normal form* (CNF) if all productions are of the form $A \to BC$ or $A \to a$.

Example. We can have the production rules:

$$S \longrightarrow AB \mid AC \mid SS, \quad C \longrightarrow SB$$

and

$$A \longrightarrow a, B \longrightarrow b$$

which is in CNF.

Definition. Let $G = (N, \Sigma, P, S)$ be a CFG. Then
(i) An *ε-production* is one of the form $A \to \varepsilon$ (for $A \in N$);
(ii) A *unit production* is one of the form $A \to B$ (for $A, B \in N$).

We first show we can modify a CFG to remove ε and unit productions but have the same defined language (apart from we might lose ε as a word in the language).

Theorem. Let G be a CFG as above. Then we can construct a CFG $\mathring{G} = (N, \Sigma, \mathring{P}, s)$ with no ε or unit productions for which

$$\mathcal{L}(\mathring{G}) = \mathcal{L}(G) \setminus \{\varepsilon\}.$$

Proof. Given the productions P of G let $\hat{P}$ be the smallest set of productions with $P \subseteq \hat{P}$ and closed under

(i) If $A \longrightarrow \alpha B \beta$ and $B \longrightarrow \varepsilon$ are both in $\hat{P}$ then $A \longrightarrow \alpha\beta$ is in $\hat{P}$.

(ii) If $A \longrightarrow B$ and $B \longrightarrow \gamma$ are both in $\hat{P}$ then $A \to \gamma$ is in $\hat{P}$.

To show that $\hat{P}$ is a finite set, start with P finite and add all types of (1) and (2) is both in P and continue. At each stage, any added production has RHS no longer than existing productions so we terminate (as finite number of possible right hand sides).

Let $\hat{G}$ be the CFG $(N, \Sigma, \hat{P}, S)$. As $P \subseteq \hat{P}$ we have that $\mathcal{L}(G) \subseteq \mathcal{L}(\hat{G})$. But for any word $w \in \mathcal{L}(\hat{G})$, a derivation of w from S in $\hat{P}$ can be replaced by the one where all the productions are obtained in the previous stage. Hence we can keep going back until we can make w with just productions in P. So $w \in \mathcal{L}(G)$, so $\mathcal{L}(G) = \mathcal{L}(\hat{G})$.

We'll now show we can remove all ε and unit productions from $\hat{P}$ to get $\mathring{P}$ without changing $\mathcal{L}(\hat{G})$ apart from losing ε. For any $w \in \mathcal{L}(\hat{G})$ with $w \neq \varepsilon$, take a minimum length derviation $S \xrightarrow[\hat{G}]{*} w$. Suppose an ε-production $B \longrightarrow \varepsilon$ is used here. So we have

$$S \xrightarrow[\hat{G}]{*} \gamma B \delta \xrightarrow[\hat{G}]{1} \gamma\delta \xrightarrow[\hat{G}]{*} w.$$

Given $\alpha \xrightarrow[\hat{G}]{1} \beta$ using (A, π) and a particular appearance of a nontemrinal X in β, say that X is *new* if it is in π.

Consider the point at which this particular B is new (B is the original S as that's replaced immediately) coming from some production $A \longrightarrow \alpha B \beta$ in $\hat{G}$,

$$S \xrightarrow{m} \eta A \theta \xrightarrow{1} \eta\alpha B \beta\theta \xrightarrow{n} \gamma B \delta \xrightarrow{1} \gamma\delta \xrightarrow{k} w$$

but $A \longrightarrow \alpha\beta$ is also in $\hat{G}$. So we can shorten the above chain to

$$\cdots \longrightarrow \eta A \theta \xrightarrow{1} \eta\alpha\beta\theta \xrightarrow{n} \gamma\delta \longrightarrow \cdots$$

as B was untouched between here. Contradiction so we can remove ε-productions.

Now say we use a unit production

$$S \xrightarrow{*} \alpha A \beta \xrightarrow{1} \alpha B \beta \xrightarrow{*} w$$

in a derivation of minimal length. Eventually as $B \notin \Sigma$ we have that B is replaced with $B \longrightarrow \gamma$. So we have

$$S \xrightarrow{m} \alpha A \beta \xrightarrow{1} \alpha B \beta \xrightarrow{n} \eta B \theta \xrightarrow{1} \eta\gamma\theta \xrightarrow{k} w \in \Sigma^*$$

and again since we have $A \longrightarrow \gamma$ in $\hat{G}$ we geta shorter derivation

$$S \xrightarrow{m} \alpha A \beta \xrightarrow{1} \alpha\gamma\beta \xrightarrow{m} \eta\gamma\theta \xrightarrow{k} w$$

which is again a contradiciton of minimality.

So throwing away all ε and unit productions from $\hat{P}$ to get $\mathring{G} = (N, \Sigma, \mathring{P}, S)$ doesn't change the language (except ε). □

Lemma. For a CFG $G = (N, \Sigma, P, S)$ without ε-productions we have a related CFG $G' = (N', \Sigma, P', S)$ with $\mathcal{L}(G') = \mathcal{L}(G)$ and such that every production is of the form $A \longrightarrow a$ or $A \longrightarrow B_1 \dots B_k$ for $k \geq 1$ where $A, B_1, \dots, B_k \in N'$ and $a \in \Sigma$.

Proof. For each $a \in \Sigma$, add a new non-terminal A_a to N to get N' and also a production $A_a \longrightarrow a$ to P. Now replace any $a \in \Sigma$ on the RHS of any production in P with A_a, unless it's of the form $B \to a$ in which case we can keep. This forms P' and hence G'.

For $\mathcal{L}(G) \subseteq \mathcal{L}(G')$, if in G we have $A \longrightarrow$ (something) which contains terminals $a_1, a_2, \dots$ then this was replaced by with A_{a_1}, A_{a_2} then apply the production rules $A_{a_i} \longrightarrow a_i$ which we added to P'.

For $\mathcal{L}(G') \subseteq \mathcal{L}(G)$, note that the new non-terminals only appear on the RHS apart from $A_a \longrightarrow a$ where they're replaced. So we can reverse the argument above thus G and G' define the same language. □

Theorem. Any CFG G gives rise to a CFG $\overline{G}$ in Chomsky normal form with $\mathcal{L}(\overline{G}) = \mathcal{L}(G) \setminus \{\varepsilon\}$.

Proof. From $G = (N, \Sigma, P, S)$ for $\mathring{G}' = (N', \Sigma, \mathring{P}', S)$ from the previous theorem and lemma together. The latter does not add ε or unit productions, so everything in $\mathring{P}'$ is of the form $A \to a$ or $A \to B_1 \dots B_k$ (with $k \geq 2$). Now for each production $A \to B_1 \dots B_k$ with $k \geq 3$ we introduce a new non-terminal C and replace the previous production with the two productions $A \to B_1 C$ and $C \to B_2 \dots B_k$. Again the language is unchanged and the maximium length of production rules in the language has decreased by 1, so we can continue this inductively until the production rules are all in Chomsky normal form. □

So given a CFG $G = (N, \Sigma, P, S)$ and $w \in \Sigma^*$, we have an algorithm to decide whether $w \in \mathcal{L}(G)$. Without loss of generality we can say that G is in CNF then it's of type 1 (since it's of type 2) and hence it is a non-contracting grammar. What about $\varepsilon \in \mathcal{L}(G)$? When we enlarge P to $\hat{P}$ we get $\varepsilon \in \mathcal{L}(\hat{G}) = \mathcal{L}(G) \iff S \to \varepsilon$ is in $\hat{P}$.

What about a CNF for G *with* $\varepsilon \in \mathcal{L}(G)$. If we convert G to CNF then naïvely we try to throw in $S \to \varepsilon$ then this could be used to obtain other words since S might appear on the right of production rules. So instead we just introduce a new start symbol $S' \in N$ and add the production rules $S' \to \varepsilon$ and $S' \to S$ and we get the same language including the empty word.

Example. Given the CFG of balanced brackets we have no unit productions but we have $S \to [S] \mid SS$ and $S \to \varepsilon$. So replace the latter with $S \to [\,]$ to get $S \to [S] \mid SS \mid [\,]$. Now add non-terminals L, R with $S \to LSR \mid SS \mid LR$ and $L \to [$ and $R \to]$. Add C and replace the first production with $S \to LC \mid SS \mid LR$ with $C \to SR$ and $L \to [$ and $R \to]$ in CNF.

3.4 Parse Trees and The Pumping Lemma for Context-Free Languages

We picture how to derive a sentence in a CFG, like a path in the transition daigram in a DFA.

Definition. (Rooted Tree) A *rooted tree* is a connected finite graph with base vertex labelled S which is simply connected.

Definition. (Parse Tree) Given a CFG G and derivation (without ε-productions) of a word $w \in \mathcal{L}(G)\backslash\{\varepsilon\}$ a *parse tree* is a rooted tree drawn downwards from S and representing the productions used in deriving w. Each vertex v is labelled with a symbol from $N \cup \Sigma$ if $\deg(v) > 1$ but from Σ if $\deg(v) = 1$. We read off the leaves from left to right. It is constructed as follows

(i) Take the first production $S \to \alpha_1$ and add $|\alpha_1|$ branches drawn from S labelled by symbols of α_1 from left to right.
(ii) For the next production, $X \to \alpha_2$ add $|\alpha_2|$ branches drawn from this X with the labelling of α_2 left to right.
(iii) Keep going until the derivation is finished and now all the leaves are labelled by terminals.
(iv) Extend these terminals down until all at the same level and then read w.

We say that the root is at level 0 and the *depth* of the free is label n of the last level.

Remark. If G is a CFG in CNF then a parse tree has $\leq 2^i$ symbols at level i as each vertex has at most 2 descendents.

Theorem. (Pumping Lemma for CFLs) Let L be a CFL. Then there exists a $n \geq 1$ such that for every word $u \in L$ with $|u| \geq n$ we can write u as 5 subwords

$$u = vwxyz$$

such that

(i) w and y are both not ε;
(ii) $1 \leq |wxy| \leq n$;
(iii) For all $k \geq 0$ we have that $vw^kxy^kz \in L$.

Proof. By previous theorem we can take a CFG, G for L in CNF. Take $n > 2^m$ for $m = |N|$ the number of non-terminals in G. If $u \in L$ and $|u| \geq n$ then any parse tree for u has depth $\geq m+1$ (else not enough symbols). Let Π be a path from the root down to the last level passing only through non-terminals except for the final vertex (i.e word back from last production), this it passes through $\geq m+1$ non-terminals and so some non-terminal occurs twice along Π. Let A be the first repeated non-terminals going from the bottom. If we use the lower A vertex as a root for the subtree below it and regard this as a smaller parse tree with start symbol A, we obtain a parse tree T_x with root A and yield x, so $A \Rightarrow^* x$. Using the upper A vertex as a root instead gives a parse tree T_{wxy} with root A and yield wxy, where w and y are the parts of the yield to the left and right of the subtree T_x (the lower A lies inside T_{wxy}). Let v and z be the parts of u to the left and right of the yield of T_{wxy}, so that $u = vwxyz$. Replacing the subtree T_x in T_{wxy} by a single leaf labelled A shows $A \Rightarrow^* wAy$. Likewise the part of the tree above the upper A gives $S \Rightarrow^* vAz$. Altogether,

$$S \Rightarrow^* vAz, \qquad A \Rightarrow^* wAy, \qquad A \Rightarrow^* x.$$

(3) By induction on k we have $A \Rightarrow^* w^kAy^k$ for all $k \geq 0$: the case $k = 0$ is trivial, and if $A \Rightarrow^* w^kAy^k$ then applying $A \Rightarrow^* wAy$ to the middle A gives $A \Rightarrow^* w^{k+1}Ay^{k+1}$. Hence for every $k \geq 0$

$$S \Rightarrow^* vAz \Rightarrow^* vw^kAy^kz \Rightarrow^* vw^kxy^kz,$$

so $vw^kxy^kz \in L$. (For $k = 0$ this is the derivation $S \Rightarrow^* vAz \Rightarrow^* vxz$.)

(1) The upper A vertex has a non-terminal descendant (the lower A), so it is not a leaf-production $A \to a$; as G is in CNF it uses a production $A \to BC$. The lower A lies in the subtree of exactly one of B, C, say B (or is B). Then the yield of the subtree at C is disjoint from x, lies entirely within w or y, and is non-empty: in CNF the only possible ε-production is $S \to \varepsilon$, and S does not appear on the right-hand side of any production, so every non-root vertex has non-empty yield. Hence $wy \neq \varepsilon$ (i.e. $|wy| \geq 1$).

(2) Going up Π from the bottom, A was the *first* repeated non-terminal. Since there are only m non-terminals, this repetition occurs among the lowest $m+1$ non-terminals of Π. So the part of Π from the upper A to the bottom contains at most $m+1$ non-terminals. Since Π was a longest path, every path in T_{wxy} has at most $m+1$ non-terminals.

Claim. If T is a parse tree in CNF in which every path from the root to a leaf passes through at most d non-terminals, then $|\text{yield}(T)| \leq 2^{d-1}$. This is by induction on d. If $d = 1$ the root uses a production $A \to a$, so the yield has length $1 = 2^0$. If $d > 1$ the root uses $A \to BC$ (or $A \to a$, which is trivial), and each of the subtrees at B and C has at most $d-1$ non-terminals on every path, so each has yield at most 2^{d-2}, giving at most $2 \cdot 2^{d-2} = 2^{d-1}$ in total.

Applying the claim to T_{wxy} with $d = m+1$ gives $|wxy| \leq 2^m < n$. Together with (1), $1 \leq |wxy| \leq n$. □

Remark. This is a different proof which differs from the lecture's proof since it was confusing and incomplete.

Example. Let $L = \{a^ib^ic^i \mid i \in \mathbb{N}\}$. Then from the above theorem, L is not a CFL. Suppose it was so whenever we write $\alpha = a^nb^nc^n$ as $vwxyz$ with $|wxy| \leq n$ and w, y not ε. Then wxy cannot contain both 'a's and 'c's. Suppose there's no 'a's so take a suitable n from the pumping lemma to get that $vw^kxy^kz \in L$ for $k \geq 2$. Now this has the same number of 'a's but certainly more 'b's and/or more 'c's. This is a contradiction, hence the language is not context free.

We can show (Example Sheet 2) that CFLs are not invariant under $\cap$ or complement.

Example. Suppose that $\sum = \{a, b, c, d\}$ and

$$L = \{w \in \Sigma^* \mid w \text{ has same number a's as c's and same number of b's as d's}\}$$

. Then suppose that L was a CFL. Let $u = a^nb^nc^nd^n$ with n a pumping number. So $u = vwxyz$ with $w, y \neq \varepsilon$ and $|wxy| \leq n$ with $vxz \in L$. So either the number of a's and/or b's changed or the number of b's and/or c's or c's and/or number of d's. Contradiction, so L is not context free.

4 Register Machines and Computatbility

The set of all functions $f : \mathbb{N} \to \mathbb{N}$ is uncountable. Which ones admit a finite description? Thus this means only countable functions have finite descriptions and most functions do not have a finite description. What actually does "description" mean. Well given a $n \in \mathbb{N}$ it should allow us to evaulate $f(n)$ by a deterministic procedure in finite time, but f could have many finite descriptions. Say we want to know if two descriptions of functions f and g describe the same function. Suppose that $f \neq g$ so we would uncover that in finite time since there is a least $m \in \mathbb{N}$ such that $f(m) \neq g(m)$ and so by iterating from $0, 1, \ldots, m$ we eventually get to m in finite time. But if they are the same function, we can never find out in finite time.

4.1 Register Machines and Computable Functions

Register machines are a simple and versatile version of the "computer code" notion (i.e they are Turing complete). They input and output natural numbers directly. Suppose we have some property of functions from $\mathbb{N} \to \mathbb{N}$ which a given function f may or may not have a formula definition of "can compute it". Given an arbitrary $S \subseteq \mathbb{N}$ we can apply this definition to S via the indicator function on S.

What about languages? $L \subseteq \Sigma^*$. Suppose that $\Sigma = \{0, 1\}$ then we can try binary. But we know that binary representations are not unique since of leading zeros. Instead if $|\Sigma| = n$, then suppose $\Sigma = \{1, \ldots, n\}$ are use "shifted" base n notation. So now $11 \in \Sigma^*$ corresponds to $n + 1 \in \mathbb{N}$ and nn corresponds to $n^2 + n$ etc. Formally

$$a_0 \ldots a_k \to a_0 + a_1 n + \ldots a_k n^k$$

but $1 \leq a_0, \ldots, a_k \leq n$. This is a bijection b, with no problems with leading zeros, and we can say that L has the property if and only if $\chi_{b(L)}$ does (where χ is the indicator function).

Definition. (Register Machine) Given discrete time steps $t = 0, 1, 2, \ldots$, a *register machine* (RM) $\mathcal{R}$ consists of

(i) A sequence of *registers* $R_1, R_2, R_3, \ldots$ where at each time t we have that $R_i(t) \in \mathbb{N}$;

(ii) We have a *finite program* F which is a fixed number of states with S_0 (terminal, S_1 (start), S_2,..., S_n. Each state has a fixed instruction performed when the machine is in state i.

- S_0: `HALT`;
- $S_i : (j, +, k)$ means add 1 to the register R_j and then move to state k with $j > 0$ and $0 \leq k \leq n$;
- $S_i : (j, -, k, \ell)$ means if $R_j \neq 0$ then subtract 1 from the register R_j and move to k. If $R_j = 0$ then just move to state ℓ with $j > 0$ and $0 \leq k, \ell \leq n$.

Definition. (Sequence of instructions) A *sequence of instructions* for a register machine $\mathcal{R}$ is the ordered list of the instructions for the program.

Definition. (Input) An *input* for $\mathcal{R}$ is, for some $k \geq 1$, a k-tuple $(n_1, \ldots, n_k) \in \mathbb{N}^k$ which are the values of $R_1, \ldots, R_k$ at the start with all others 0. At $t = 0$ we start in state S_1 then follow stopping when/if we reach the `HALT` state S_0.

Definition. (Program Diagram) A *program diagram* for $\mathcal{R}$ is a finite directed graph with vertices the state sand labelled arrowe for the instructions. We take an arrow with label $R_j + 1$ for the instructions $(j, +, k)$ and the arrow with label $R_j - 1$ for the instruction $(j, -, k, \ell)$ with a dotted arrow to state ℓ.

Example. The transfer register machine which copies R_i to R_j and sets R_i to zero. This can be done by emptying R_j first and then cycles between two states where we add one to R_j then remove one from R_i and repeat this until R_i is zero where we then go to S_0 and `HALT`.

Definition. (Halting) For some $k > 0$ a program $\mathcal{R}$ *halts* on an input $(n_1, \ldots, n_k) \in \mathbb{N}^k$ if it reaches state S_0 in finite time written $\mathcal{R}(n_1, \ldots, n_k)\downarrow$. If it never halts we write that $\mathcal{R}(n_1, \ldots, n_k)\uparrow$. The *halting set* $\Omega(\mathcal{R})$ is a set of inputs on which $\mathcal{R}$ halts

$$\Omega(\mathcal{R}) = \bigcup_{k>0} \{(n_1, \ldots, n_k) \in \mathbb{N}^k \mid \mathcal{R}(n_1, \ldots, n_k)\downarrow\}.$$

Remark. $(n_1, \ldots, n_k)$ and $(n_1, \ldots, n_k, 0)$ are the same input. The *output* will be the value of $R_1 \in \mathbb{N}$ when the machine halts.

Definition. For a program $\mathcal{R}$, the *upper register index* $\text{upper}(\mathcal{R})$ is the largest index of a register appearing in instructions for $\mathcal{R}$ (which is finite since the instructions are finite).

For input in $\mathbb{N}^k$, think of $\text{upper}(\mathcal{R}) > k$ so we have our first k input registers and then some registers we use for computing.

Definition. (Partial Function) A *partial function* $f : \mathbb{N}^k \dashrightarrow \mathbb{N}$ is one where $\text{dom}(f) \subseteq \mathbb{N}^k$ and is undefined otherwise. We say that f is *partial computable* by a program P if for all $(n_1, \ldots, n_k)$ in the domain of f, we have that $P(n_1, \ldots, n_k)\downarrow$ with $f(n_1, \ldots, n_k)$ equalling the value of R_1 or halting but if $P(n_1, \ldots, n_k)\uparrow$ then f is undefined at $(n_1, \ldots, n_k)$. Thus given P and $k > 0$ then we obtain a partial function $f : \mathbb{N}^k \to \mathbb{N}$.

Lemma. We can add R_i to R_j $(i \neq j)$ leaving R_i unchanged.

Proof. Do it yourself. □

In fact the function obtained from this register machine in the proof corresponds to the partial function $(n_1, n_2) \to n_1 + n_2$ which is defined on the whole of $\mathbb{N}^2$ so it is total. Likewise the "emptying" function $n \to 0$ is also total on $\mathbb{N}$.

Corollary. We can copy the contents of the register R_i to R_j, leaving R_j unchanged.

Proof. First we empty R_j and then add R_i to R_j, so done. □

Remark. The idea of this proof is using *subroutines* by forming a program by combining with another problem that proceeds it. We will never have any fighting between subroutines since we just allocate registers to be owned by a single subroutine only by reindexing.

4.2 Partial Recursive Functions

Partial computable functions have good closure properties.

Theorem. The following total fuctions are computable:
(i) For each $i \leq k$ we have the *projection function* $(n_1, \ldots, n_k) \to n_i$;
(ii) The zero and successor function $n \to 0$ and $n \to n + 1$.

These are called the *basic functions*. We also have that:

(iii) If $f : \mathbb{N}^k \dashrightarrow \mathbb{N}$ and $g_1, \ldots, g_k : \mathbb{N}^\ell \dashrightarrow \mathbb{N}$ are partial computable then so is $h(n_1, \ldots, n_\ell) = f(g_1(n_1, \ldots, n_\ell), \ldots, g_k(n_1, \ldots, n_\ell))$ where defined (if all are total functions then h is also total);

(iv) (Recursion) If f on $k \geq 0$ variables and g on $k+2$ variables are partial computable then so is the partial function $h : \mathbb{N}^{k+1} \dashrightarrow \mathbb{N}$ defined inductively by

$$h(n_1, \ldots, n_k, 0) = f(n_1, \ldots, n_k)$$

and

$$h(n_1, \ldots, n_k, n_{k+1} + 1) = g(n_1, \ldots, n_{k+1}, h(n_1, \ldots, n_{k+1})).$$

Again if f and g are total then h is also.

(v) (Minimisation) If f with $k+1$ variables is partially computable then so $g : \mathbb{N}^k \dashrightarrow$ given by "finds the first zero as the last place varies", so

$$g(n_1, \ldots, n_k) = \begin{cases} n & \text{if } f(n_1, \ldots, n_k, n) = 0 \text{ and } f(n_1, \ldots, n_k, m) > 0 \ \forall m < n \\ \text{undefined} & \text{otherwise} \end{cases}.$$

However here we don't have that f total means g is total.

Proof.

(i) Previous example, just transfer R_i to R_1.

(ii) Empty R_1 to get $n \to 0$. To get $n \to n+1$ just add 1.

(iii) Use subroutines. First transfer $R_1, \ldots, R_\ell$ to $R_{N+1}, \ldots, R_{N+\ell}$ for N sufficiently large. Then for each $1 \leq i \leq \ell$ in turn copy $R_{N+1}, \ldots, R_{N+\ell}$ to $R_{k+1}, \ldots, R_{k+\ell}$. Then perform g_i with all registers shifted up by k and transfer the answer from R_{k+1} to R_i. Then empty $R_{k+2}, \ldots, R_N$ and apply f.

(iv) Ommitted.

(v) Ommitted. □

Definition. (Partial Recursive Function) The class of *partial recursive functions* is the smallest class of partial functions $f : \mathbb{N}^k \dashrightarrow \mathbb{N}$ over all $k \geq 1$ which are closed under the operations in the theorem above. That is f can be constructed from basic functions and use of (iii), (iv), (v), finitely many times.

Example. If $k = 1$, then $n \to n$ (projection) is partial recursive and constants $n \to c$ via $n \to 0$ and composing the successor function c times.

So the previous theorem says that partial recursive $\implies$ partial computable.

Definition. (Primitive Recursive) A function f is *primitive recursive* if minimisation is never used in constructing f.

So primitive recursive $\implies$ total recursive.

Example. Addition and multiplication are primitive recursive. Addition is done by recursion on the successor function and multiplication is then done by recursion on the addition function. Similarly by the same reasoning the function $(m, n) \to m^n$ is also primitive recursive.

Definition. Write p_i for the ith prime (with $p_0 = 2$). For $n > 0$ write $(n)_i$ for the largest power of $p_i \mid n$.

Lemma. For fixed i, the function $n \to (n)_i$ is primitive recursive.

Proof. We'll prove this is stages. First note that for any finite sequence $m_0, \ldots, m_s \in \mathbb{N}^{s+1}$ then the function

$$h(n) = \begin{cases} m_n & n \leq s \\ 0 & n > s \end{cases}$$

is primitive recursive by induction on s and recursion. For $k = 0$, if $g : \mathbb{N}^2 \to N$ primitive recursive then so is $h(0) = c$, $h(n+1) = g(n, h(n))$. The inductve step starts with a given $f : \mathbb{N} \to \mathbb{N}$ primitive recursive, $g(n, m) = f(n)$ is primitive recursive by projection and composition. Thus $h(0) = c$, $h(n+1) = f(n)$ where $f(n)$ is primitive recursive (or $f(h(n))$ is primitive recursive).This includes (for each fixed $k \geq 2$) the functions

(i) The step function

$$\text{Step}_k(n) = \begin{cases} 1 & 0 \leq n \leq k-2 \\ 0 & n > k-2 \end{cases}.$$

(ii) The delta function

$$\delta_k(n) = \begin{cases} 1 & n = k \\ 0 & n \neq k \end{cases}.$$

(iii) The epsilon function

$$\varepsilon(n) = \delta_0(\delta_0(n)) = \begin{cases} 0 & n = 0 \\ 1 & n > 0 \end{cases}.$$

(iv) The slope function

$$\text{Slope}_k(n) = \begin{cases} n+1 & 0 \leq n \leq k-2 \\ 0 & n > k-2 \end{cases}.$$

Now we can show the remainder function is primitive recursive, where $\text{Rem}_k(n) = n \bmod k$. Recursion as above, $h(0) = 0$, $h(n+1) = f(h(n))$ primitive recursive if f is. Here $\text{Rem}_k(n+1) = \text{Slope}_k(\text{Rem}_k(n))$. We can show by Example Sheet 3 that these are all primitive recursive. We can also therefore show the floor and divide functions are primitive recursive, given by

$$\text{Floor}_k(n) = \left\lfloor \frac{n}{k} \right\rfloor \qquad \text{Divide}_k(n) = \begin{cases} \frac{n}{k} & n \equiv 0 \bmod k \\ 0 & \text{otherwise} \end{cases}.$$

Now we can show that division by powers

$$\text{Powers}_k(n, m) = \begin{cases} \frac{n}{k^m} & n \equiv 0 \bmod k^m \\ 0 & \text{otherwise} \end{cases}$$

and the function

$$\text{Maxpow}_k(n) = \begin{cases} 0 & n = 0 \\ \text{largest power of } k \text{ dividing} & n \neq 0 \end{cases}.$$

We first define (by recursion) $h(n, 0) = 0$ and $h(n, m + 1) = h(n, m) + \varepsilon(\text{Power}_k(n, m + 1))$ which is therefore primitive recursive since addition is primitive recursive. With a bit more effort in things like this we can prove that $\text{Maxpower}_k(n)$ is primitive recursive, and hence since $(n)_i = \text{Maxpower}_{p_i}(n)$ this is also primitive recursive for fixed i. □We use this to encode $k$tuples in $\mathbb{N}^k$ as elements of $\mathbb{N}$ and functions too. Say $F : \mathbb{N}^{r+1} \to \mathbb{N}^{r+1}$ with $F = (f_0, \ldots, f_r)$ primitive recursive. Given $n \in N$ obtain $\mathbf{n} = ((n)_0, \ldots, (n)_r) \in \mathbb{N}^{r+1}$, apply F then encode. Obtain the encoded $\overline{F} : \mathbb{N} \to \mathbb{N}$ as $\overline{F}(n) = 2^{f_0(\mathbf{n})} \ldots p_r^{f_r(\mathbf{n})}$ which is primitive recursive if F is from the previous work.

4.3 Partial Computable $\iff$ Partial Recursive

We just saw that all partial recursive functions are partial computable, now we have a big theorem for the converse.

Theorem. Every partial computable function $f : \mathbb{N}^k \dashrightarrow \mathbb{N}$ is partial recursive.

Proof. From a program P for f, we'll define $g : \mathbb{N}^{k+2} \to \mathbb{N}$ as what is happening in P i.e $g(n_1, n_2, \ldots, n_k, 0, t)$ is the state of P at time t with input $(n_1, \ldots, n_k)$. Note that $t = 0$ gives 1 and if we halt at $t = t_0$ then we get that for all $t \geq t_0$, $g(n_1, \ldots, n_k, i, t)$ is the contents of R_i at time t. So we have $M = \text{upper}(P)$ with $g(n_1, \ldots, n_k, i, t) = 0$ for all $i > M$. Now g is a total function. Suppose that g is primitive recursive. Define $q(n_1, \ldots, n_k)$ as the smallest t such that $g(n_1, \ldots, n_k, 0, t) = 0$ i.e it is the number of steps taken to halt on this input (if it does). Then q is partial recursive if g is by minimisation and hence

$$f(n_1, \ldots, n_k) = g(n_1, \ldots, n_k, 1, q(n_1, \ldots, n_k))$$

is partial recursive. So it's enough to prove the following claim.

Claim. g as defined above is primitive recursive.

Proof. We will obtain g from the transition function σ which computes the changes for each instruction, but we use the encoded $\overline{\sigma}$, so $\sigma : \mathbb{N}^{M+1} \to \mathbb{N}^{M+1}$ is

$$\sigma(i, n_1, \ldots, n_M) = (\text{new state, new values in the registers})$$

and this is unchanged if $i >$ largest state. If the instruction is $S_i(j, +, k)$ then

$$\sigma(i, n_1, \ldots, n_M) = (k, n_1, \ldots, n_j + 1, \ldots, n_M).$$

Each component function of σ is primitive recursive, hence σ is primitive recursive, so the encoded $\overline{\sigma}$ is also primitive recursive by the previous conservation. We can now recover g through $h : \mathbb{N}^{k+2} \to \mathbb{N}$. This is like g but starting in any statem though with the i variable encoded. Thus $h(n_0, n_1, \ldots, n_k, t)$ is a coded integer of state and registers $R_1, \ldots, R_m$ of P at time t for input $n_1, \ldots, n_k$ and start state n_0. Formally define

$$h(n_0, n_1, \ldots, n_k, 0) = 2^{n_0} 3^{n_1} \ldots p_k^{n_k} p_{k+1}^0 \ldots p_m^0$$

and

$$h(n_0, n_1, \ldots, n_k, t+1) = \overline{\sigma}(h(n_0, \ldots, n_k, t))$$

thus h is primitive recursive (by recursion as $\overline{\sigma}$ is primitive recursive). Finally

$$g(n_0, \ldots, n_k, i, t) = h(1, n_1, \ldots, n_k, t)$$

Finally

$$g(n_1, \ldots, n_k, i, t) = (h(1, n_1, \ldots, n_k, t))_m$$

for $0 \leq m \leq M$ where the RHS is primitive recurisve for fixed m and 0 if $m > M$. Now

$$g(n_1, \ldots, n_k, i, t) = \sum_{j=0}^{M} \delta_j(m) h(1, n_1, \ldots, n_k, t)$$

which is primitive recursive.

4.4 Algorithms and Recursive Sets

Definition. (Recursive/Computable) A (partial) function $f : \mathbb{N}^k \dashrightarrow \mathbb{N}$ is *recursive* or *computable* if it's total recurisve.
A set $X \subseteq \mathbb{N}^k$ is *recursive* or *computable* or *decidable* if the character function

$$_X(n) = \begin{cases} 1 & n \in X \\ 0 & n \notin X \end{cases}$$

is a computable function.

Definition. (Algorithm) An algorithm is any process taking an input $\mathbb{N}^k$ for some k or a recursive subset $X \subseteq \mathbb{N}^k$ and giving an output $\mathbb{N}$ which is simulated by a register/Turing machine.